

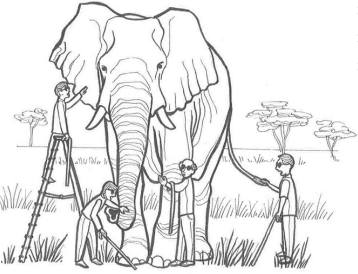
NESNEYE YÖNELİK PROGRAMLAMA

UML ile Analiz ve Tasarım

Emir Öztürk

MODELLEME

- Farklı açıdan inceleme
- Tanımlama
- Analiz / Tasarım



Modelleme bir sistemi farklı açılardan inceleyip bir tanımlama oluşturulmasıdır. Analiz aşamasında modelleme yapıldığında tasarımın bir taslağı oluşturulmuş olur. Böylece oluşabilecek sorunlar tasarım üzerinde incelenebilir. Modelleme yapılması ile farklı gruplar arasındaki hata oranının azaltılması amaçlanır. Örneğin müşteri ile yazılım ekipleri arasındaki iletişimden doğacak bir hatayı veya yazılım grupları arasında olan iletişimde çıkabilecek hataları ortadan kaldırmak için ortak bir model tasarlanması oldukça faydalı olacaktır.

UML*

- Unified Modeling Language
- Genel amaçlı modelleme dili
 - Programlama dili
- Sistemin dizaynını görselleştirmek
- Görselleştirme standardı belirlemek



UML bir yazılım sürecinin tüm aşamaları için modelleme standardı sunar. Yazılım geliştirme sürecindeki tüm grupların ortak bir görsel model üzerinde çalışmasını sağlayarak iletişim problemlerini en aza indirger. Yazılımın tamamı hakkında bir model üretildiği için geliştirilme aşamasından önce tüm kararların alınması yazılım sürecine yardımcı olmaktadır. UML nesneye yönelik yapıya dayanmaktadır. UML'in asıl geliştirilme amacı farklı notasyonlarda geliştirilmiş görsel modelleme yaklaşımlarına bir standart getirmektir. UML bir programlama dili değildir.

USE-CASE DİYAGRAMI

- Aktörler
- Durumlar
- Sistem
- Sırasız
- Include / Extend



Sistemde kişilerin kullandığı durumlar açıklanır.

Mevcut sistemde tüm aktiviteler tanımlandıktan sonra bu aktivitelere hangi aktörlerin erişebileceği belirlenir.

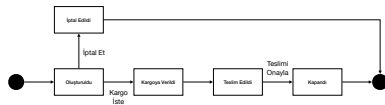
Herhangi bir sıra belirtilmez. Birbirine bağlı gözüken iki işlem, bu işlemlerin birbirinden sonra gelmesini değil, aralarındaki içeme veya genişletme ilişkisini belirler.

İçeme ilişkisi birden fazla aktivitenin içerdiği ortak özellikleri tek bir noktada toplamak için kullanılır.

Genişletme ilişkisi ile yapılabilecek aktiviteye bağlı seçimlik başka bir aktivite varsa bunu ifade eder.

DURUM MAKİNESİ DİYAGRAMLARI

- Graflar
- Düğümler
- Kenarlar
- Alt durumlar



Durum makinesi diyagramları nesnelerin belirli bir durumdaki davranışını ifade eder.

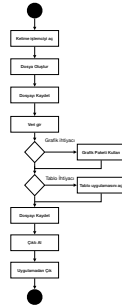
Durum makinesi diyagramları graf (çizge) şeklinde gösterilirler.

Düğümmler nesnelerin durumlarını gösterirken kenarlar ise bu durumların arasındaki geçişi ifade etmektedir.

Aynı zamanda bir durum kendi içerisinde bir durum makinesi diyagramı olabilir ve alt durumlar da bu şekilde gösterilebilirler.

AKTİVİTE DİYAGRAMLARI

- Akış şeması
 - Algoritma
- Aktivite diyagramı
 - Aktiviteler arası bağlantı



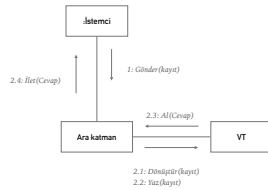
Aktivite diyagramı akış şemalarının aktivite boyutunda kullanılması ile oluşturulur. Akış şemalarında her adımda algoritma detayı verilirken aktivite diyagramlarında aktivitelerin akışı ve aktiviteler arası geçiş koşulları verilmektedir. Hangi aktivitenin hangi durumda çağırılacağı ya da aktivitelerin sonuçlarına göre hangi aktivitelerin çalıştırılacağı belirtilmektedir.

ETKİLEŞİM DİYAGRAMLARI

- Sıra diyagramı
- İletişim diyagramı
- Etkileşim genel bakış diyagramı
- Zamanlama diyagramı

İLETİŞİM DİYAGRAMLARI

- Nesnelerin iletişimi
- Nesneler ve mesajlar



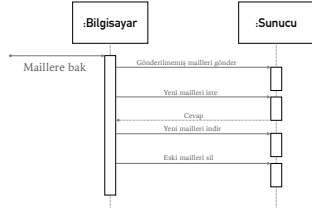
Nesnelerin iletişimini gösterir.

Nesnelerin, birbirine gönderdiği mesajları ifade eder.

Mesajlar iletilirken bir sıra numarasına sahiptir. Aynı istekte gönderilen ardışık mesajlar aynı sıra numarası ile başlar virgülden sonra numara verilmeye devam edilir.

SIRA DİYAGRAMI

- Zaman odaklı
- İşlem sırası
- Yüksek seviyeli iletişim



İşlemlerin nasıl gerçekleştirildiğini gösterir. Sıra diyagramları zaman odaklıdır ve işlem sıraları yatay ekseninde gösterilir.

Yüksek seviyeli bir şekilde aktif nesneler arası iletişimi gösterir.

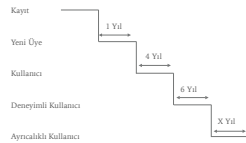
Nesneler, mesajlar ve zaman eksenlerinden oluşurlar. Nesnelerden aşağı doğru çizilen kesikli çizgiler zamanı göstermektedir. Çizgilerin nesnelere olan en uzak noktası daha geç zamanı ifade etmektedirler.

Aktivitelerden çıkan oklar mesajları ifade eder. Bir nesnenin hangi zamanda diğer nesne ile iletişime geçtiğini bu oklar göstermektedir.

Zaman eksenleri ise okların alınmasından itibaren bir işlemin süresini ifade eden ve zaman ekseninde aşağı doğru uzayan kutulardır.

ZAMANLAMA DİYAGRAMI

- Sınıflar veya durumlar arası ilişki
- İlişkilerin zaman karşılıkları



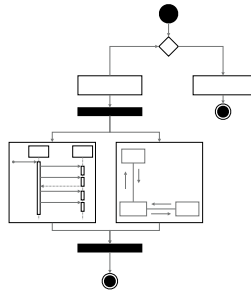
Sınıflar arasındaki ilişkiyi zaman odaklı göstermek için kullanılır.

Lineer bir zaman ekseninde koşulların değişimini gösterir.

Sistemde durumların birbiri ile olan ilişkisini (durumdan duruma geçiş) gösterirken aynı zamanda bu geçişlerin zamanını da belirtir.

ETKİLEŞİM GENEL BAKIŞ DİYAGRAMLARI

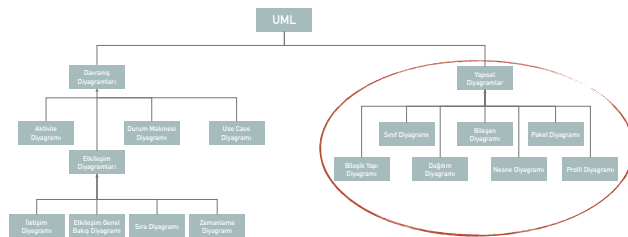
- Akış şeması
- Etkileşimler
- Etkileşimler arası bağlantılar



Akış şeması şeklindedir. Akış şemalarının aralarındaki düğümler diğer etkileşim diyagramlarından oluşur.

Düğüm etkileşimleri gösterirken düğümler arası bağlantılar bu etkileşimlerin hangi durumda tetikleneceğini belirtir. Amaç etkileşimlere ve aralarındaki bağlantılara genel bir bakış sağlamaktır.

UML DİYAGRAMLARI



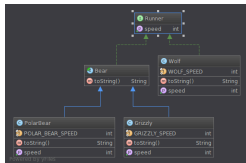
Yapısal diyagramlarda modellenen sistemde nelerin var olması gerektiği vurgulanır.

Davranış diyagramlarında modellenen sistemde nelerin meydana gelmesi gerektiğini belirtir.

Davranış diyagramlarının bir alt kümesi olan Etkileşim diyagramlarında ise modellenen sistemdeki elemanlar arasındaki veri ve komut akışı gösterilir.

SINIF DİYAGRAMLARI*

- Sistemdeki sınıflar
- Sınıfların özellikleri
- Sınıfların metotları
- Nesnelerin birbirleri arasındaki ilişkiler
- Veri modelleri



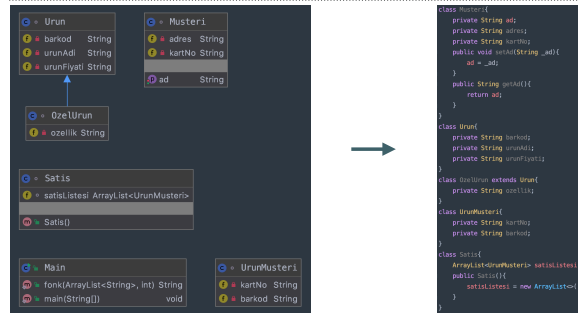
Sistemde bulunan sınıfları özellikleri, metotları ve birbirleri arasındaki ilişkileri tanımlarlar. Sınıf hiyerarşisi de bu diyagramlardan görülebilmektedirler.

Aynı zamanda özellik ve metotların detaylarını da içerirler.

Veri modelleme için de kullanılabilirler.

Sınıflar üç alanlı bir kutu ile gösterilirler. İlk alan sınıfın adını, ikinci alan özellikleri ve son alan da metotları içermektedir.

SINIF DİYAGRAMLARI*



Bir sınıf diyagramına karşılık gelen Java kodu

```
class Musteri {
    private String adres;
    private String kartNo;
    private String ad;
    public void setAd(String _ad) {
        ad = _ad;
    }
    public String getAd() {
        return ad;
    }
}

class Urun {
    private String barkod;
    private String urunAdi;
    private String urunFiyati;
}

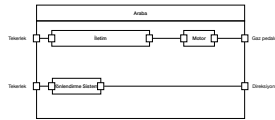
class ÖzelUrun extends Urun {
    private String ozelik;
}

class Satış {
    private ArrayList<UrunMusteri> satilistesi;
    public void satis() {
        satilistesi = new ArrayList<>();
    }
}

class Main {
    public static void fonk(ArrayList<String> _urunlar, int _musteriNo) {
        // ...
    }
    public static void main(String[] args) {
        // ...
    }
}
```

BİLEŞİK YAPI DİYAGRAMI

- Sınıf diyagramı işlevi
- Sınıf, arayüz ya da paketin iç yapısı
- İç yapıdaki ilişkiler
- Sistemin tümü yerine bir bileşene odaklanma



Bir sınıfın, arayüzün veya bir paketin iç yapısını belirtmek için kullanılır.

Sistemin genel görüntüsünün yerine sınıf odaklı yapıyı göstermeyi amaçlar

Aynı zamanda bir sınıfın değişken olarak içerdiği diğer sınıfları ve bu sınıflarla olan ilişkiyi de gösterir.

Bileşik yapı diyagramı bir sınıf diyagramı ile aynı role sahiptir fakat sınıfların iç yapısını daha detaylı göstermektedir.

NESNE DİYAGRAMI

- Sınıf
- Nesne
- Örnek (Instance)
- Anlık durum

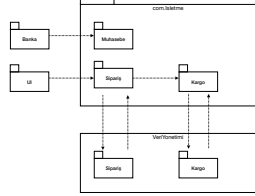


Nesneler sınıflardan oluşturulmuş örneklerdir.

Sistemin çalışma anında belirli bir zamanda tanımlanmış nesnelerin o anki durumunu göstermek amacı ile nesne diyagramları kullanılır. Bu sebeple sınıf diyagramlarına benzerler.

PAKET DİYAGRAMI

- Model elementlerinin düzenlenmesi
- Büyük ölçekli proje
- Yüksek seviyeli
- Karmaşıklığın azaltılması



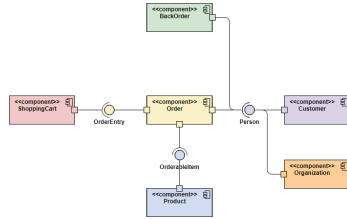
Paket diyagramı büyük ölçekte projelerin düzenlenmesi ve organizasyonu için kullanılır. Yapıları ve alt modüller arası ilişkileri gösterir.

Paket diyagramı karmaşık diyagramları basitleştirmek için kullanılır. Sınıfları paketler şeklinde gruplar.

Paketler mantıksal olarak birbiri ile alakalı UML elemanlarından oluşur.

BİLEŞEN DİYAGRAMLARI

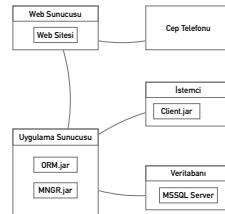
- Bileşen
- Bileşenler arası bağlantılar
- Alt komponentler
- Karmaşık sistemlerin açıklanması



Bileşen bir yazılımın bir işi yapan parçalarıdır. Bu bileşenlerin tamamının birbirleri ile ilişkileri de dahil olmak üzere bir şema üzerinde verilmesi ile bileşen diyagramları oluşturulmaktadır. Bileşen diyagramlarının amacı uygulamanın fonksiyonelliğinin tespitinde bulunmak ve aynı zamanda uygulama geliştirme planı yapmaya yardımcı olmaktır.

DAĞITIM DİYAGRAMI

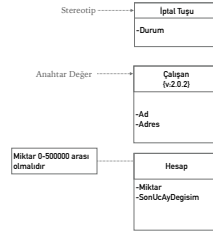
- Parçaların fiziksel dağılımı
- Döğümler ve döğümler arası bağlantı
- Döğümler üzerinde bulunan parçalar



Yazılımın parçalarının fiziksel olarak nasıl dağıtıldığını gösterir. Bir sistemin fiziksel olarak hangi döğümlere sahip olduğunu ve hangi döğümde yazılımın hangi modülünün çalıştırıldığını gösterir. Aynı zamanda bu döğümler arasındaki bağlantıları da ifade eder.

PROFİLLER

- UML Genel tanımları
- Özel alanlar
- Özel teknolojiler
- Stereotipler
 - Yeni element ekleme
- Etiketler
 - Mevcut modelleri genişletme
- Kısıtlamalar
 - Her zaman sağlanması gereken işlevler



UML spesifik bir alan ya da uygulama için tanımlanmamıştır fakat böyle bir uygulama kullanılması durumunda UML'in bu ekstra durumları belirtmemesi modelleme işlemini zorlaştırmaktadır. Böyle durumlarda profil diyagramı UML modelleri üzerinde bir genişletme mekanizması sunmaktadır.

Profiller standart kuralları bozmadan stereotipler, etiketli değer tanımları ve kısıtlamalar kullanılması ile oluşturulurlar.

Profiller herhangi bir iş alanı ya da herhangi bir platform için kullanılabilirler. Stereotipler UML'in sözlüğünü geliştirmek için kullanılırlar. Yeni model elementleri eklenebilir.

Etiketli değerler mevcut modelleri genişletme işlemi gerçekleştirirler. Örneğin bir sınıfta anahtar değer ikilileri olarak version, 1.1.0 değerleri saklanabilir. Bu sayede test ekibi hangi versiyonun kullanıldığını modelden görebilirler.

Kısıtlamalar her zaman sağlanması gereken işlevleri tanımlar. Örneğin gerçek zamanlı bir sistemde her bileşenin ne kadar sürede cevap vermesi gerektiği bir kısıt olarak tanımlanabilmektedir.

KAYNAKLAR

- https://en.wikipedia.org/wiki/Unified_Modeling_Language
- <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-composite-structure-diagram/>
- http://web.firat.edu.tr/jaydin/bmu112/week_6_uml.pdf
- https://en.wikipedia.org/wiki/UML_Partners
- <http://www.csharpnedit.com/articles/read/?id=6>
- <https://www.slideshare.net/aselmaib/uml-ile-modelleme>
- https://web.itu.edu.tr/bkurt/Courses/goose/ooswe_release_1_0_6.pdf
- http://web.firat.edu.tr/jaydin/bmu112/week_6_uml.pdf
- <https://bidb.itu.edu.tr/sevir-defteri/blog/2013/09/08/s%C4%B1n%C4%B1f-diyagramlar%C4%B1>
- <https://www.ahmetkaymaz.com/2009/09/14/uml-ve-uml-diyagramlari-i/>
- <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-profile-diagram/>
- <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-object-diagram/>
- https://www.wikiwand.com/en/Sequence_diagram